

Conditionally borrowed ranges

Document #: P2017R1
Date: 2020-02-19
Project: Programming Language C++
Audience: LWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction and Motivation](#)
- [3 History and Status Quo](#)
 - [3.1 Implementation Strategy](#)
- [4 Proposal](#)
- [5 Wording](#)
- [6 Implementation](#)
- [7 References](#)

§ 1 Revision History

R0 to R1: `safe_range` was renamed to `borrowed_range` in Prague by resolving [\[LWG3379\]](#). This paper has been updated to reflect that.

§ 2 Introduction and Motivation

Consider the following approach to trimming a `std::string`:

```
auto trim(std::string const& s) {  
    auto isalpha = [](unsigned char c){ return std::isalpha(c); };  
    auto b = ranges::find_if(s, isalpha);  
    auto e = ranges::find_if(s | views::reverse, isalpha).base();  
    return subrange(b, e);  
}
```

This is a fairly nice and, importantly, safe way to implement `trim`. The iterators `b` and `e` returned from `find_if` will not dangle, since they point into the string `s` whose lifetime outlives the function.

Except this code will not compile at the moment, either in C++20 or in range-v3, failing on the declaration of `e`. The algorithm `find_if` is in 25.5.5 [\[alg.find\]](#) is declared as:

```
template<input_range R, class Proj = identity,  
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
```

```
constexpr borrowed_iterator_t<R>
ranges::find_if(R&& r, Pred pred, Proj proj = {});
```

R will deduce as `reverse_view<ref_view<std::string const>>`, which does *not* satisfy `borrowed_range` (it is neither an lvalue reference, nor does `reverse_view` currently opt-in to being a `borrowed_range`) hence the return type `borrowed_iterator_t<R>` is the type dangling rather than being the type `iterator_t<R>`. Instead of getting the reverse iterator we might have expected, that we need to call `.base()` on, we get effectively nothing. We are forced to rewrite the above as:

```
auto trim(std::string const& s) {
    auto isalpha = [](unsigned char c){ return std::isalpha(c); };
    auto b = ranges::find_if(s, isalpha);
    auto reversed = s | views::reverse;
    auto e = ranges::find_if(reversed, isalpha).base();
    return subrange(b, e);
}
```

Which is an unnecessary code indirection. The goal of this paper is to make the initial example just work. We clearly have a borrowed range that is not marked as such, so I consider this to be a library defect.

§ 3 History and Status Quo

Ranges introduced with it the concept *forwarding-range*. This was then renamed to `safe_range` by [P1870R1] in Belfast, and again to `borrowed_range` by resolving [LWG3379] in Prague, but the core concept remains the same. A range is a `borrowed_range` when you can hold onto its iterators after the range goes out of scope. There are two kinds of borrowed ranges:

- lvalue references to ranges are always borrowed ranges. It's not the lvalue reference itself which owns the data, so if the reference dies, we're fine.
- ranges which must necessarily opt in to being considered borrowed, by way of the new `enable_borrowed_range` variable template, which defaults to `false` for all types.

There are several borrowed ranges which do this opt in today:

- `ref_view` is a borrowed range. It's basically a reference - the iterators it gives out are iterators to the range it refers to, which it has no ownership of. If the `ref_view` dies, the referred to range can still be around.
- `string_view` is a borrowed range. Like `ref_view`, it just refers to data - the iterators it gives out are iterators into some other containers. `span` and `subrange` are similar.
- `empty_view` doesn't even have any data, so it's trivially borrowed.
- `iota_view` works by having the iterators themselves own the "counter", so having the iterators stick around is sufficient.

And that's it. We have six, *unconditionally* borrowed ranges. All other ranges and views in the standard library are *unconditionally* not borrowed. But this is far too strict. As the opening example demonstrates, there are many more kinds of borrowed ranges you can construct than *just* the chosen six.

This issue was first pointed out by Johel Ernesto Guerrero Peña in [\[stl2.640\]](#).

§ 3.1 Implementation Strategy

A range is going to be borrowed if its iterators do not in any way refer to it. For the ranges in the working draft which are unconditionally borrowed, this follows directly from how they actually work. But for some other ranges, it might depend on implementation strategy.

One can imagine different specifications for views like `transform_view` and even `filter_view` that might allow them to be borrowed, but that's beyond the scope of this paper which is limited to those views that are already borrowed as specified.

§ 4 Proposal

Several range adapters semantically behave as if they have a single member of some templated view type. If that underlying view type is a `borrowed_range`, the range adapter itself can be transitively borrowed. For example, `s | views::reverse` has the type `reverse_view<ref_view<string const>>`. This can be a `borrowed_range` because `ref_view<string const>` is a `borrowed_range`. Likewise, `s | views::reverse | views::take(3)` can also be a `borrowed_range` by extending this logic further.

Here is a table of all the range adapters and factories in the current working draft, what their current `borrowed_range` status is, and what this paper proposes.

Name	Current Status	Proposed
<code>empty</code>	Borrowed	No change
<code>single</code>	Not Borrowed	No change. It's the view which holds the element, not the iterators.
<code>iota</code>	Borrowed	No change.
<code>istream</code>	Not Borrowed	No change. The iterators need to refer to parent view, which holds onto the element.
<code>ref</code>	Borrowed	No change.
<code>filter</code>	Not Borrowed	No change. The view needs to own the predicate.
<code>transform</code>	Not Borrowed	No change, as above.
<code>take</code>	Not Borrowed	Conditionally borrowed, based on the underlying view. The iterators are just iterators into the underlying view (or thin wrappers thereof)..
<code>take_while</code>	Not Borrowed	No change, same as <code>filter</code> .
<code>drop</code>	Not Borrowed	Conditionally borrowed, same as <code>take</code>

drop_while	Not Borrowed	Conditionally borrowed. Unlike take_while or filter, we only need the predicate to find the new begin. Once we found it, it's just transparent.
join	Not Borrowed	No change. This one is quite complex and iterators need to refer the join_view.
split	Not Borrowed	No change, as with join.
counted	Not actually its own view, counted(r, n) is actually either some subrange or ill-formed, so it's already borrowed.	
common	Not Borrowed	Conditionally borrowed based on the underlying view. All it does is propagate iterators.
reverse	Not Borrowed	Conditionally borrowed based on the underlying view. All it does is propagate reverse iterators.
elements/keys/values	Not Borrowed	Conditionally borrowed based on the underlying view. This is a special case of transform_view where the transform is actually encoded into the type, so it doesn't need to be held onto by the view itself.

§ 5 Wording

Add six variable template specializations to 24.2 [\[ranges.syn\]](#):

```
#include <initializer_list>
#include <iterator>

namespace std::ranges {
    // [...]

    // [range.take], take view
    template<view> class take_view;

+ template<class T>
+ inline constexpr bool enable_borrowed_range<take_view<T>> =
    enable_borrowed_range<T>;

    namespace views { inline constexpr unspecified take = unspecified; }

    // [...]

    // [range.drop], drop view
    template<view V>
        class drop_view;

+ template<class T>
+ inline constexpr bool enable_borrowed_range<drop_view<T>> =
    enable_borrowed_range<T>;

    namespace views { inline constexpr unspecified drop = unspecified; }
```

```

// [range.drop.while], drop while view
template<view V, class Pred>
    requires input_range<V> && is_object_v<Pred> &&
        indirect_unary_predicate<const Pred, iterator_t<V>>
    class drop_while_view;

+ template<class T, class Pred>
+     inline constexpr bool enable_borrowed_range<drop_while_view<T, Pred>> =
        enable_borrowed_range<T>;

namespace views { inline constexpr unspecified drop_while = unspecified; }

// [...]

// [range.common], common view
template<view V>
    requires (!common_range<V> && copyable<iterator_t<V>>)
    class common_view;

+ template<class T>
+     inline constexpr bool enable_borrowed_range<common_view<T>> =
        enable_borrowed_range<T>;

namespace views { inline constexpr unspecified common = unspecified; }

// [range.reverse], reverse view
template<view V>
    requires bidirectional_range<V>
    class reverse_view;

+ template<class T>
+     inline constexpr bool enable_borrowed_range<reverse_view<T>> =
        enable_borrowed_range<T>;

namespace views { inline constexpr unspecified reverse = unspecified; }

// [range.elements], elements view
template<input_range V, size_t N>
    requires see below;
    class elements_view;

+ template<class T, size_t N>
+     inline constexpr bool enable_borrowed_range<elements_view<T, N>> =
        enable_borrowed_range<T>;

template<class R>
    using keys_view = elements_view<all_view<R>, 0>;
template<class R>
    using values_view = elements_view<all_view<R>, 1>;

namespace views {
    template<size_t N>
        inline constexpr unspecified elements = unspecified ;
        inline constexpr unspecified keys = unspecified ;
        inline constexpr unspecified values = unspecified ;

```

```
}  
}
```

§ 6 Implementation

This has been implemented in range-v3 [[range-v3.1405](#)]. The PR includes the six range adapters in this paper, along with a smattering of other range adapters from range-v3 that can also be made conditionally borrowed in this manner (`const`, `chunk`, `delimit`, `drop_exactly`, `indirect`, `intersperse`, `move`, `slice`, `sliding`, `tail`, `trim`, `unbounded`, and `zip`) as well as a bunch of other range adapters that can be additionally made conditionally borrowed based on both the underlying range and the shape of invocables that they rely on (`group_by`, all the `set_algorithm_view` adapters, `split_when`, `take_while/iter_take_while`, `transform`, and `zip_view/iter_zip_view`).

§ 7 References

[LWG3379] Casey Carter. “safe” in several library names is misleading.

<https://wg21.link/lwg3379>

[P1870R1] Barry Revzin. 2019. forwarding-range is too subtle.

<https://wg21.link/p1870r1>

[range-v3.1405] Barry Revzin. 2020. Making more range adapters safe.

<https://github.com/ericniebler/range-v3/pull/1405>

[stl2.640] Johel Ernesto Guerrero Peña. 2019. Unsafe views that are actually safe.

<https://github.com/ericniebler/stl2/issues/640>